Anlea Maria Jose, Roll NO.: 02, ASDS Lab Exp 4, Date: 27-02-2026

```
#importing libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

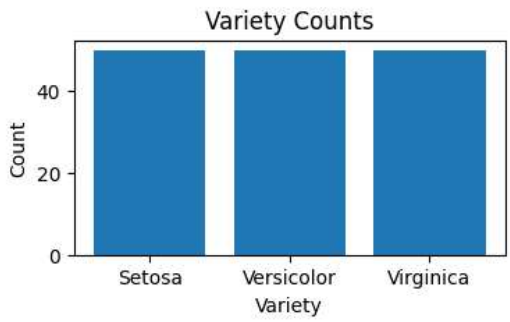
```
#Reading iris.csv and getting basic details
df = pd.read_csv("iris.csv")
df.info()
df.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

Next steps: Generate code with df | New interactive sheet

```
#Plotting bar graph of variety
variety_counts = df['variety'].value_counts()
plt.figure(figsize=(4, 2))
plt.bar(variety_counts.index, variety_counts.values)
plt.title('Variety Counts')
plt.xlabel('Variety')
plt.ylabel('Count')
plt.show()
```




```
# Compute measures of central tendency of sepal.length
sepal_length = df["sepal.length"]
mean_value = sepal_length.mean()
median_value = sepal_length.median()
mode_value = sepal_length.mode()[0]


print("Mean   :", round(mean_value, 3))
print("Median :", round(median_value, 3))
print("Mode   :", round(mode_value, 3))


# Determine skewness type
if mean_value > median_value>mode_value:
    skewness = "Positively Skewed"
```

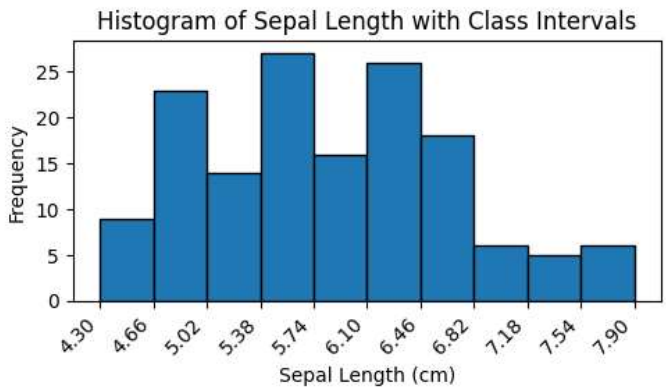
```
elif mean_value < median_value<mode_value:
    skewness = "Negatively Skewed"
else:
    skewness = "Symmetric Distribution"


print("\nType of Skewness:", skewness)
```

```
Mean   : 5.843
Median : 5.8
Mode   : 5.0

Type of Skewness: Positively Skewed
```

```
#Creating continuous frequency distribution of sepal.length
num_bins = 10 # 10 class intervals

# Calculate frequency distribution and bin edges
counts, bin_edges = np.histogram(df['sepal.length'], bins=num_bins)

# Create class intervals labels
bin_labels = [f"{bin_edges[i]:.2f} - {bin_edges[i+1]:.2f}" for i in range(len(bin_edges)-1)]

# Create a DataFrame for the frequency table
frequency_df = pd.DataFrame({'Class Interval': bin_labels, 'Frequency': counts})

print("Frequency Distribution of Sepal Length:")
display(frequency_df)
```

Frequency Distribution of Sepal Length:

	Class Interval	Frequency
0	4.30 - 4.66	9
1	4.66 - 5.02	23
2	5.02 - 5.38	14
3	5.38 - 5.74	27
4	5.74 - 6.10	16
5	6.10 - 6.46	26
6	6.46 - 6.82	18
7	6.82 - 7.18	6
8	7.18 - 7.54	5
9	7.54 - 7.90	6

Next steps: Generate code with frequency_df New interactive sheet

```
# Plot histogram using the calculated bins
plt.figure(figsize=(5, 3))
plt.hist(df['sepal.length'], bins=bin_edges, edgecolor='black')
plt.title('Histogram of Sepal Length with Class Intervals')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Frequency')
plt.xticks(bin_edges, rotation=45, ha='right')
# plt.grid(axis='y', alpha=0.75)
plt.tight_layout()
plt.show()
```